

RGB Fix (Scroll Lock)

Ubuntu 26.04 + GNOME (Wayland) — Generic USB Gaming Keyboard

This document keeps a record of everything that was done to fix the keyboard's RGB issue, which used to be triggered by the Scroll Lock key and stopped working on Ubuntu 26.04. Keep this file: if the RGB stops working again (for example after replacing the keyboard or updating the system), come back here to reapply or understand the solution.

The Problem

The gaming keyboard kit (generic, semi-mechanical) uses a trick common in cheap keyboards: the RGB effect is triggered by the keyboard's own firmware whenever the computer turns on the standard Scroll Lock LED. This normally works on Windows and also in Linux's plain text mode (TTY), because the kernel turns that LED on directly. However, GNOME, when running on Wayland (the default on Ubuntu 26.04, since GNOME dropped Xorg support starting with version 49), does not automatically sync that LED with the hardware. Result: pressing Scroll Lock did nothing.

The Solution

We created a small Python program that runs continuously in the background, listening to the keyboard directly. Every time the Scroll Lock key is pressed, the program manually writes to the system file that controls that LED, turning the RGB on or off. This program was configured as a systemd service, meaning it starts on its own every time the computer boots, with no manual steps required.

Files created on the system: - Python script:
/usr/local/bin/scrolllock-rgb.py - systemd service:
/etc/systemd/system/scrolllock-rgb.service

1. The Python Script — scrolllock-rgb.py

This is the program that runs the whole time and makes the RGB work. Below is the complete code, split into parts, with an explanation of each section right after it.

1.1 — Imports

```
#!/usr/bin/env python3
import evdev
from evdev import Import ecodes
import glob
import time
```

The first line tells Linux that this file should be run with Python 3. `evdev` is the library that allows reading keyboard/mouse events directly from the system (it was installed with the command `sudo apt install python3-evdev`). `ecodes` provides the standardized codes for each key (e.g., the code for the Scroll Lock key). `glob` is used to search for files by a name pattern (used to find the LED automatically). `time` is used for small pauses within the program.

1.2 — Function that finds the Scroll Lock LED

```
def find_led_path():
    matches = glob.glob("/sys/class/leds/*scrolllock*")
    if matches:
        return matches[0] + "/brightness"
    return None
```

Linux represents the Scroll Lock LED as a file inside `/sys/class/leds/`, but the exact number (for example `input2::scrolllock` or `input24::scrolllock`) changes every time the computer boots, depending on the order in which Linux detects the USB devices. So, instead of hardcoding that number in the code (which broke the script after a reboot), this function searches, at the moment the program starts, for any folder containing “scrolllock” in its name and uses it automatically.

1.3 — Function that finds the correct keyboard

```
def find_keyboard():
    for path in evdev.list_devices():
        dev = evdev.InputDevice(path)
        caps = dev.capabilities().get(ecodes.EV_KEY, [])
        if ecodes.KEY_SCROLLLOCK in caps and ecodes.KEY_A in caps:
            return dev
    return None
```

The computer may have several input devices (keyboard, mouse, media buttons, etc.). This function goes through all connected devices and checks which one is able to send both the Scroll Lock key and the A key — that is, which one is the actual keyboard, and not some secondary device (such as the “Consumer Control” or “System Control” devices that appear separately on the same USB keyboard).

1.4 — Preparation before starting to listen

```
LED_PATH = find_led_path()
device = find_keyboard()

if LED_PATH is None or device is None:
    raise SystemExit("Keyboard or LED not found yet")

state = 0
```

Here the program calls the two functions above to find the LED path and identify the keyboard. If for some reason either one isn’t found (for example, if the script tries to run too quickly, before Linux finishes recognizing the keyboard at boot), the program stops with an error message — and `systemd` (explained in section 2) will automatically try to run it again until it succeeds. The state variable stores whether the RGB is currently on (1) or off (0).

1.5 — Function that turns the LED on/off

```
def set_led(value):  
    with open(LED_PATH, "w") as f:  
        f.write(str(value))
```

This function simply opens the LED file we found earlier and writes the number 0 (off) or 1 (on) to it. It has the exact same effect as manually running the command `echo 1 | sudo tee /sys/class/leds/.../brightness`, which we used at the very beginning to test.

1.6 — The main loop (the program's "heart")

```
for event in device.read_loop():  
    if event.type == ecodes.EV_KEY and event.code ==  
ecodes.KEY_SCROLLLOCK and event.value == 1:  
        state = 1 - state  
        set_led(state)  
    elif event.type == ecodes.EV_LED:  
        time.sleep(0.05)  
        set_led(state)
```

This is the part that runs forever, waiting for keyboard events. It does two things:

- **When you press Scroll Lock** (`event.value == 1` means "key pressed," not "released"): the program flips the stored state (if it was 0 it becomes 1, if it was 1 it becomes 0) and sends this new value to the LED, turning the RGB on or off.
 - **When GNOME changes any LED** (`EV_LED`, which happens when you press Caps Lock or Num Lock): the keyboard receives a packet from GNOME with the state of all 3 LEDs together, and this would unintentionally turn off our Scroll Lock. So, whenever this happens, the script waits 0.05 seconds (enough time for GNOME to finish writing) and forces the LED back to the stored state.
-

2. The systemd Service — scrolllock-rgb.service

systemd is Ubuntu's system responsible for automatically starting programs, including during boot. We created a "service" so that the Python script above runs on its own, with no terminal needed, every time the computer boots.

```
[Unit]  
Description=Syncs the Scroll Lock LED with the keyboard's RGB  
After=multi-user.target  
  
[Service]  
ExecStart=/usr/bin/python3 /usr/local/bin/scrolllock-rgb.py  
Restart=always  
RestartSec=2  
User=root  
  
[Install]  
WantedBy=multi-user.target
```

Section/Key	Explanation
[Unit]	Section with general information about the service.
Description=...	A descriptive name, shown when you check the service's status.
After=multi-user.target	Tells systemd to only try starting this service after the system's basic multi-user environment is already up (avoids trying to run too early).
[Service]	Section describing how the program should be run.
ExecStart=...	The exact command systemd should run: calls Python 3 pointing to our script.
Restart=always	If the program closes or errors out for any reason, systemd should start it again automatically.
RestartSec=2	Waits 2 seconds before trying to restart. This mattered because, right after boot, the keyboard had sometimes not yet been recognized by Linux — the script would fail, wait 2 seconds, try again, until it worked.
User=root	Runs the program as administrator (root), because writing to the LED file requires that permission.
[Install]	Section that says how the service connects to the system's boot process.
WantedBy=multi-user.target	Makes the service start together with the operating system, every time the computer is turned on.

3. All Commands Used in the Terminal

Below is every command used while solving the problem, in the order they were run, listed one by one with an explanation of what each one does.

Command 1

```
echo $XDG_SESSION_TYPE
```

Shows whether the current graphical session is running on Wayland or Xorg (X11). We used this to find out that the system was on Wayland, which explained why the LED wasn't syncing.

Command 2

```
lsusb
```

Lists all connected USB devices, including the keyboard, along with the manufacturer's name. Helps identify the connected hardware.

Command 3

```
ls /usr/share/xsessions/
```

Lists the Xorg-mode graphical sessions installed on the system. This returned a "folder does not exist" error because, starting with Ubuntu 25.10/26.04, GNOME completely dropped Xorg support.

Command 4

```
sudo apt update
```

Updates the list of packages available for installation, to make sure the following install commands would fetch the latest versions.

Command 5

```
sudo apt install xserver-xorg gnome-session
```

An attempt to install the Xorg graphical server and the GNOME session. This turned out to be a dead end, since GNOME 49 no longer offers an Xorg session.

Command 6

```
dpkg -L gnome-session | grep xsessions
```

Lists all files installed by the gnome-session package and filters only the ones related to graphical sessions. Used to confirm that the package didn't include any Xorg session, only Wayland.

Command 7

```
ls /sys/class/leds/ | grep -i scroll
```

Searches inside the system's LED folder for any entry with "scroll" in the name. This is how we found the exact file that controls the Scroll Lock LED (and the RGB along with it).

Command 8

```
echo 1 | sudo tee /sys/class/leds/input24::scrolllock/brightness
```

Writes the value 1 directly to the LED file, manually turning the RGB on. This was the test that confirmed the problem could be solved by writing directly to this file, without relying on GNOME.

Command 9

```
cat /proc/bus/input/devices | grep -A 5 -i keyboard
```

Lists the input devices (keyboard, mouse, etc.) recognized by Linux, showing name, identification, and which "event" file corresponds to each one. Used to figure out which event belonged to the actual keyboard.

Command 10

```
sudo apt install python3-evdev
```

Installs the Python library that allows reading keyboard events directly from the system. It's used inside the scrolllock-rgb.py script.

Command 11

```
sudo mkdir -p /usr/local/bin && sudo tee /usr/local/bin/scrolllock-rgb.py > /dev/null << 'EOF' ... EOF
```

Creates the /usr/local/bin folder (if it doesn't already exist) and then creates the Python script file with all the content between the EOF markers, without needing to open any interactive text editor.

Command 12

```
sudo chmod +x /usr/local/bin/scrolllock-rgb.py
```

Grants execute permission to the script, allowing it to be run directly as a program.

Command 13

```
sudo python3 /usr/local/bin/scrolllock-rgb.py
```

Runs the script manually in the terminal, to test whether it works before making it automatic. While it runs, the terminal stays busy waiting for keyboard events (Ctrl+C to stop).

Command 14

```
sudo tee /etc/systemd/system/scrolllock-rgb.service > /dev/null << 'EOF' ... EOF
```

Creates the systemd service configuration file, which teaches Ubuntu to run the script on its own, including on subsequent boots.

Command 15

```
sudo systemctl daemon-reload
```

Tells systemd that new or modified service files have been created, so it re-reads the configurations.

Command 16

```
sudo systemctl enable scrolllock-rgb.service
```

Marks the service to start automatically every time the computer boots.

Command 17

```
sudo systemctl start scrolllock-rgb.service
```

Starts the service immediately, without needing to restart the computer.

Command 18

```
sudo systemctl restart scrolllock-rgb.service
```

Restarts the service, useful after changing the script, so the new version comes into use.

Command 19

```
systemctl status scrolllock-rgb.service --no-pager
```

Shows whether the service is active (active/running), for how long, and the latest log lines — without locking the terminal into a pager mode (--no-pager).

Command 20

```
systemctl is-enabled scrolllock-rgb.service
```

Quickly confirms whether the service is set to start on boot (expected answer: “enabled”).

Command 21

```
journalctl -u scrolllock-rgb.service --no-pager
```

Shows the complete history of messages (logs) generated by this specific service. Useful for investigating errors in case the RGB stops working again in the future.

Command 22

```
sudo reboot
```

Fully restarts the computer. Used to test whether the service really starts on its own, with no manual command, after powering on the PC.

End of document. Keep this file — if the RGB stops responding to Scroll Lock in the future, start by reviewing section 3, beginning with the status and log commands (systemctl status and journalctl).